Nama: Hana Maftuha

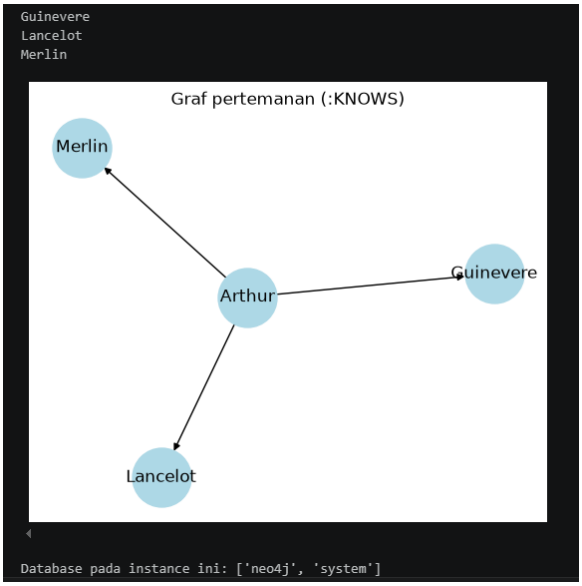
NIM: 24031554066

Kelas: 2024B

UTS GRAFF SAINS DATA

1. Pertanyaan Pendahuluan (Checkpoint)
    a. Apa nama database yang digunakan pada tutorial ini?
    Jawab: (parameter database_=”neo4j”)
    b. Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!
    Jawab: Untuk mengakses database lain dalam satu instance, ada beberapa cara yang dapat digunakan:
    a. Parameter database_ pada driver.execute_query(...) (atau database= pada driver.session(...)) di Python, untuk menentukan database tujuan. Contohnya, pada praktikum ini saya menjalankan driver.execute_query("SHOW DATABASES", database_="system") untuk mengakses database system.
    b. Klausa USE <nama_database> di awal query Cypher, misalnya USE system SHOW DATABASES atau USE neo4j MATCH (n) RETURN n.
    c. Perintah: use <nama_database> pada Neo4j Browser untuk berpindah database yang sedang aktif.
    d. Perintah DDL untuk administrasi database yang dijalankan pada database system, yaitu SHOW DATABASES, CREATE DATABASE <nama>, DROP DATABASE <nama>, START DATABASE <nama>, dan STOP DATABASE <nama>.
    e. Perintah DML seperti MATCH, MERGE, CREATE, dan DELETE untuk mengelola data di dalam database data, misalnya neo4j.
    Sebagai catatan, pada Neo4j edisi Community, satu instance hanya mendukung satu database data (neo4j) ditambah system, sehingga perintah CREATE DATABASE untuk membuat database baru hanya dapat dipakai pada edisi Enterprise. Hal ini sesuai dengan hasil pada praktikum, yaitu hanya terdapat neo4j dan system.
    c. Apa jenis graf pertemanan yang telah dibuat disini?
    Jawab: Graf pertemanan yang dibuat pada tutorial ini adalah graf berarah (directed graph). Graf ini terdiri dari empat simpul berlabel Person, yaitu Arthur, Guinevere, Lancelot, dan Merlin, serta tiga relasi berarah berjenis KNOWS, yaitu Arthur Guinevere, Arthur Lancelot, dan Arthur Merlin. Bentuknya menyerupai graf bintang (star graph) dengan Arthur sebagai pusatnya (hub).

Graf ini juga bersifat tidak berbobot karena relasi KNOWS tidak memiliki properti apa pun, dan bersifat homogen karena semua simpulnya memiliki label yang sama, yaitu Person. Karena arah relasinya dari Arthur menuju temannya, hubungan ini bersifat satu arah. Artinya, Guinevere, Lancelot, dan Merlin tidak tercatat mengenal Arthur, kecuali ada relasi yang dibuat dengan arah sebaliknya. Selain itu, perintah MERGE yang digunakan memastikan simpul dan relasi hanya dibuat jika belum ada, sehingga tidak terjadi duplikasi walaupun fungsi add_friend dijalankan berulang kali.



Gambar 1

2. Soal UTS

2.1 Wrangling dan EDA

a. Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yang dipilih tiap mahasiswa tidak boleh sama! (akan ditentukan) Misal: Fold training (P1) → 01/2023 - 03/2023; Fold testing (P2) 01/2024 - 06/2024.

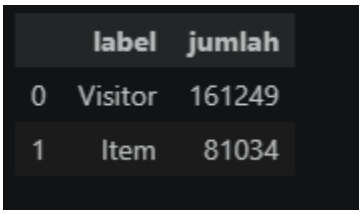
b. Terapkan proses filter ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!

Jawab: Pada soal ini saya menggunakan dua periode waktu sebagai filter data interaksi, yaitu P1 (fold training): 16 sampai 30 Juni 2015 dan P2 (fold testing): 01 sampai 18 September 2015. Kedua periode ini sesuai dengan kombinasi yang ditetapkan untuk saya. Data asli events.csv mencakup 3 Mei 2015 sampai 18 September 2015 dengan total 2.756.101 baris, sehingga kedua periode tersebut berada di dalam rentang data yang tersedia.

c. Lakukan query untuk mendapatkan komposisi dari subgraph yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!

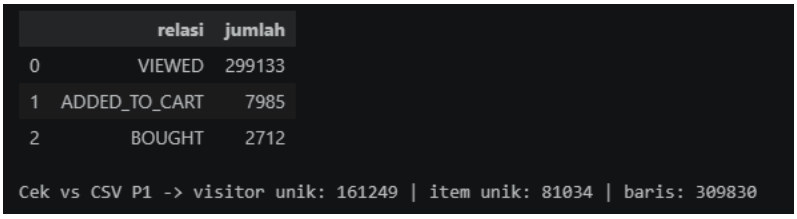
Jawab:

- Jenis simpul dan Jumlah masing-masing simpul



	label	jumlah
0	Visitor	161249
1	Item	81034

Gambar 2



	relasi	jumlah
0	VIEWED	299133
1	ADDED_TO_CART	7985
2	BOUGHT	2712

Cek vs CSV P1 -> visitor unik: 161249 | item unik: 81034 | baris: 309830

Gambar 3

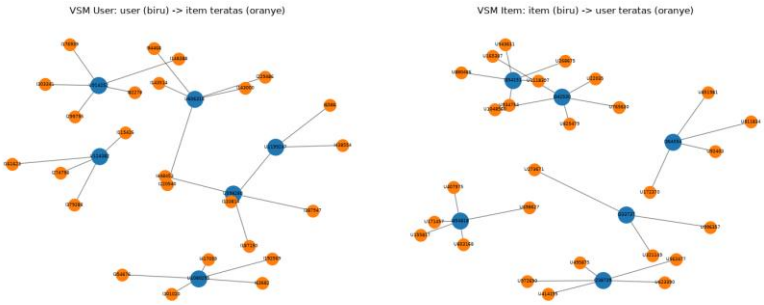
- Jelaskan CYPHER SQL yang digunakan!
  CYPHER SQL yang digunakan, yaitu:
  1. `MATCH (n)` digunakan untuk mengambil semua simpul yang ada di graf.
  2. `labels(n)[0]` digunakan untuk mengambil label dari simpul (`Visitor` atau `Item`), sedangkan `count(n)` menghitung jumlah simpulnya. Karena `label` merupakan kolom non-agregat, Neo4j otomatis mengelompokkan hasilnya per label, mirip dengan `GROUP BY` pada SQL.
  3. `MATCH ()-[r]->()` digunakan untuk mengambil semua relasi berarah, `type(r)` mengambil jenis relasinya, dan `count(r)` menghitung jumlahnya.
  4. `ORDER BY jumlah DESC` digunakan untuk mengurutkan hasil dari jumlah yang paling banyak.

d. Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!

Jawab: Pada VSM User, setiap visitor direpresentasikan sebagai vektor pada ruang dimensi item. Seluruh interaksi diubah menjadi matriks User × Item (X) dalam bentuk sparse (CSR). Isi setiap selnya adalah bobot implicit feedback yang dijumlahkan untuk setiap pasangan visitor dan item, dengan bobot VIEWED = 1, ADDED_TO_CART = 3, dan BOUGHT = 5. Distribusi item per user sangat timpang. Median item per user hanya 1, persentil ke-90 hanya 2 item, persentil ke-99 sebanyak 7 item, dan nilai maksimumnya 874 item. Hal ini menunjukkan bahwa sebagian besar user hanya berinteraksi dengan 1 sampai 2 item. Visualisasi sampel VSM User menggunakan networkx dapat dilihat pada Gambar 4.

e. Lakukan VSM terhadap simpul yang diasumsikan sebagai Item. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!

Jawab: Pada VSM Item, setiap item direpresentasikan sebagai vektor pada ruang dimensi user, yaitu hasil transpose dari matriks sebelumnya (X^T). Median user per item adalah 1, persentil ke-90 sebanyak 6, persentil ke-99 sebanyak 24, dan nilai maksimumnya 257. Jumlah non-zero dan sparsity sama dengan VSM User karena datanya sama, hanya orientasi matriksnya yang berbeda. Meskipun begitu, vektor item rata-rata sedikit lebih terisi dibandingkan vektor user (2,96 dibanding 1,49). Visualisasi sampel VSM Item dapat dilihat pada Gambar 4.



Gambar 4

f. Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!

Jawab: Dari matriks biner B, terdapat dua kemungkinan proyeksi graf, yaitu User–User ($B \cdot B^T$) dan Item–Item ($B^T \cdot B$). Untuk membandingkannya, saya menghitung batas atas jumlah pasangan non-nol (Σ derajat^2) tanpa harus membentuk matriks hasil proyeksinya.

proyeksi	dimensi	ukuran dense (GB, float32)	batas atas pasangan non-nol	batas atas density
User-User ($B \cdot B^T$)	161249 x 161249	104.004960	3185732.0	0.000123
Item-Item ($B^T \cdot B$)	81034 x 81034	26.266037	4734104.0	0.000721

Gambar 5

Proyeksi yang dipilih, yaitu USER-USER. Karena dimensi user = 161,249 vs 81,034; estimasi pasangan non-nol 3,185,732 vs 4,734,104. Proyeksi dengan dimensi lebih kecil dan pasangan non-nol lebih sedikit lebih efisien (memori/komputasi) dan biasanya lebih stabil (tiap vektor punya lebih banyak data pendukung).

g. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan Python

Jawab: Similarity dihitung menggunakan cosine similarity pada vektor biner proyeksi User–User. Record acuan dipilih secara acak menggunakan np.random.default_rng(42) dari visitor yang berderajat minimal 5, dan terpilih Visitor 124914 (derajat 6).

Record acuan: Visitor visitorId=124914 (derajat=6)

	relasi	jumlah
0	ADDED_TO_CART	7985
1	BOUGHT	2712
2	VIEWED	299133

Gambar 6

- Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendiri record acuannya)!

  Jawab:

```
# No. 7a. Cypher top-5
REL_T = "VIEWED|ADDED_TO_CART|BOUGHT"
cypher_top5 = f"""
MATCH (a:{L} {{{K}: $ref}})-[:{REL_T}]-(x)-[:{REL_T}]-(b:{L})
WHERE a <> b
WITH a, b, count(DISTINCT x) AS common
MATCH (a)-[:{REL_T}]-(xa) WITH a, b, common, count(DISTINCT xa) AS da
MATCH (b)-[:{REL_T}]-(xb) WITH a, b, common, da, count(DISTINCT xb) AS db
RETURN b.{K} AS id, common, round(common / sqrt(da * db), 4) AS cosine
ORDER BY cosine DESC, common DESC
LIMIT 5
"""
print(cypher_top5)
top5_cypher = df_q(cypher_top5, ref=ref_id)
display(top5_cypher)
```

Gambar 7

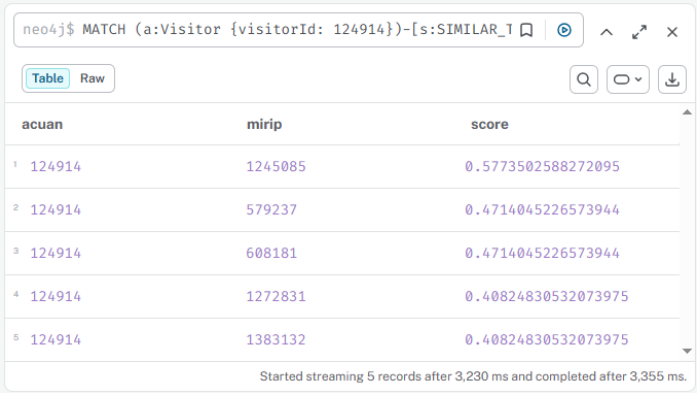
	id	common	cosine
0	1245085	2	0.5774
1	608181	2	0.4714
2	579237	2	0.4714
3	165844	1	0.4082
4	1036049	1	0.4082

Gambar 8

Perhitungan menggunakan Python (matriks sparse dengan normalisasi L2) menghasilkan nilai yang sama. Perbedaannya hanya pada peringkat 4 dan 5 (visitor 1383132 dan 1272831), karena nilainya seri di 0,4082 sehingga urutan antar-visitor yang seri bisa berbeda.

- Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil querynya!

  Jawab:



Gambar 9

```
UNWIND $rows AS r
MATCH (a:{L} {{{K}: r.a}}), (b:{L} {{{K}: r.b}})
MERGE (a)-[s:SIMILAR_TO]->(b)
SET s.score = r.score, s.metric = 'cosine_binary'
```

Gambar 10

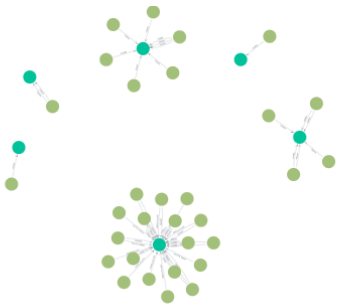
	acuan	mirip	score
0	124914	1245085	0.577350
1	124914	579237	0.471405
2	124914	608181	0.471405
3	124914	1272831	0.408248
4	124914	1383132	0.408248

Gambar 11

h. Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!

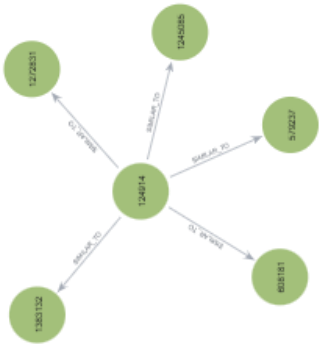
Jawab:

Sebelum



Gambar 12

Sesudah



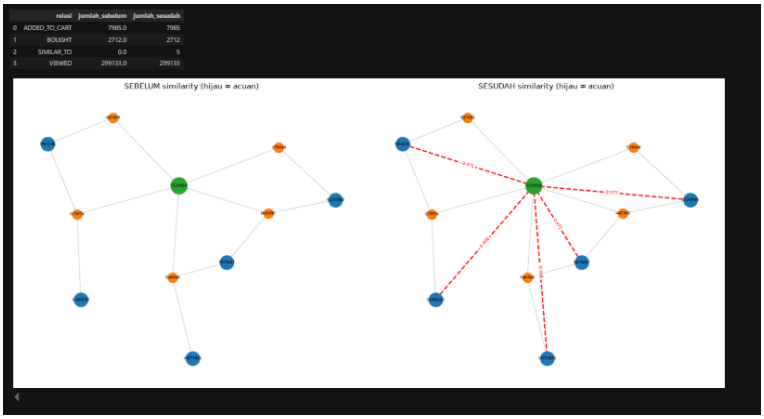
Gambar 13

```
# No. 8
sesudah = df_q("MATCH ()-[r]->() RETURN type(r) AS relasi, count(r) AS jumlah ORDER BY relasi")
cmp_rel = sebelum.merge(sesudah, on="relasi", how="outer", suffixes=("_sebelum", "_sesudah")).fillna(0)
display(cmp_rel)

def ego_graph(with_similar, max_oth=8):
    nodes = [ref_idx] + [int(j) for j in top_idx if sims[j] > 0]
    ref_cols = set(Mb[ref_idx].indices)
    cnt = {}
    for e in nodes[1:]:
        for c in set(Mb[e].indices) & ref_cols:
            cnt[c] = cnt.get(c, 0) + 1
    others = [c for c, _ in sorted(cnt.items(), key=lambda t: -t[1])[:max_oth]]
    if not others:
        others = list(ref_cols)[:max_oth]
    pre_e, pre_o = L[0], ("I" if L == "Visitor" else "V")
    G = nx.Graph()
    for e in nodes:
        G.add_node(f"{pre_e}{int(ent_ids[e])}")
        cols = set(Mb[e].indices)
        for c in others:
            if c in cols:
                oth_id = int(iid[c]) if L == "Visitor" else int(uid[c])
                G.add_edge(f"{pre_e}{int(ent_ids[e])}", f"{pre_o}{oth_id}", kind="interaksi")
    sim_edges = []
    if with_similar:
        for j in top_idx:
            if sims[j] > 0:
                sim_edges.append((f"{pre_e}{ref_id}", f"{pre_e}{int(ent_ids[j])}", round(float(sims[j]), 3)))
    return G, sim_edges, pre_e

fig, axes = plt.subplots(1, 2, figsize=(16, 7))
for ax, flag, judul in [(axes[0], False, "SEBELUM similarity"), (axes[1], True, "SESUDAH similarity")]:
    G, sim_edges, pre_e = ego_graph(flag)
    for a, b, _ in sim_edges:
        G.add_node(a); G.add_node(b)
    pos = nx.spring_layout(G, seed=SEED, k=0.9)
    ent_nodes = [n for n in G if n.startswith(pre_e)]
    oth_nodes = [n for n in G if not n.startswith(pre_e)]
    nx.draw_networkx_nodes(G, pos, nodelist=ent_nodes, node_color="tab:blue", node_size=500, ax=ax)
    nx.draw_networkx_nodes(G, pos, nodelist=[f"{pre_e}{ref_id}"], node_color="tab:green", node_size=700, ax=ax)
    nx.draw_networkx_nodes(G, pos, nodelist=oth_nodes, node_color="tab:orange", node_size=250, ax=ax)
    nx.draw_networkx_edges(G, pos, alpha=0.4, edge_color="gray", ax=ax)
    nx.draw_networkx_labels(G, pos, font_size=6, ax=ax)
    if sim_edges:
        nx.draw_networkx_edges(G, pos, edgelist=[(a, b) for a, b, _ in sim_edges], edge_color="red",
                               style="dashed", width=2, ax=ax)
        nx.draw_networkx_edge_labels(G, pos, edge_labels={(a, b): s for a, b, s in sim_edges},
                                     font_color="red", font_size=7, ax=ax)
    ax.set_title(f"{judul} (hijau = acuan)")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Gambar 14



Gambar 15

2.2 Analytics

a. Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu

- tuliskan skrip Python dalam merekomendasikan KNN-nya!

  Jawab: Untuk memilih user secara acak dan reproducible, saya membuat daftar kandidat berupa visitor yang ada di data training (P1) dan juga memiliki interaksi di P2, yaitu sebanyak 2.307 user. Daftar ini diurutkan terlebih dahulu, kemudian diacak menggunakan np.random.default_rng(42). User pertama yang menghasilkan minimal 3 item rekomendasi dipilih, sehingga hasilnya selalu sama setiap kali kode dijalankan ulang. Algoritma rekomendasi KNN yang saya gunakan adalah sebagai berikut:

  1. Menghitung cosine similarity antara user target dan semua user pada matriks berbobot X.
  2. Mengambil k = 5 tetangga terdekat yang memiliki similarity lebih dari 0.
  3. Menghitung skor tiap item, yaitu jumlah dari (similarity tetangga × bobot interaksi tetangga pada item tersebut).
  4. Membuang item yang sudah pernah diinteraksi oleh user target. Mengurutkan item dari skor tertinggi.

```python
# No. 1a
K_NN = 5
N_ADS = 3
Xn = normalize(X, norm="l2")
XnT = Xn.T.tocsr()

def recommend_knn(user_id, k=K_NN, n_rec=5):
    ui = uid_index.get_loc(user_id)
    sim = (Xn[ui] @ XnT).toarray().ravel()
    sim[ui] = 0.0
    nn = np.argpartition(-sim, k)[:k]
    nn = nn[np.argsort(-sim[nn])]
    nn = nn[sim[nn] > 0]
    if len(nn) == 0:
        return nn, sim, []
    skor = (sp.csr_matrix(sim[nn].reshape(1, -1)) @ X[nn]).toarray().ravel()
    skor[X[ui].indices] = 0.0
    top = np.argsort(-skor)[:n_rec]
    top = top[skor[top] > 0]
    return nn, sim, [(int(iid[j]), float(skor[j])) for j in top]

GT_EVENTS = ["view", "addtocart", "transaction"]
test = pd.read_csv(P2_CSV)
test = test[test["event"].isin(GT_EVENTS)]
gt = test.groupby("visitorid")["itemid"].apply(set).to_dict()

kandidat_user = sorted(u for u in gt if u in uid_index)
print("Kandidat user (ada di P1 dan P2):", len(kandidat_user))

rng_user = np.random.default_rng(SEED)
urutan = rng_user.permutation(kandidat_user)
target, hasil = None, None
for u in urutan[:500]:
    u = int(u)
    nn, sim, recs = recommend_knn(u)
    if len(recs) >= N_ADS:
        target, hasil = u, (nn, sim, recs)
        break
assert target is not None, "Tidak ada user valid; coba ganti SEED atau periode."
nn, sim, recs = hasil
print(f"User terpilih (SEED={SEED}): visitorid = {target}")
print("Jumlah item di training:", int(X[uid_index.get_loc(target)].nnz))
```

Gambar 16

- berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!
  Jawab: User yang terpilih adalah visitor 21682, yang memiliki 2 item pada data training. Tetangga KNN-nya adalah sebagai berikut.

5 tetangga terdekat (KNN):

	visitorid	cosine
0	758601	0.7071
1	1021667	0.6325
2	10053	0.5000
3	1394234	0.4082

Gambar 17

Walaupun k = 5, hanya 4 tetangga yang memiliki cosine positif, sehingga rekomendasi dihitung dari 4 tetangga tersebut. Hasil rekomendasinya adalah sebagai berikut.

Kandidat rekomendasi (top-5), 3 teratas = item yang diiklankan:

rank	itemid	skor
1	230397	0.632456
2	180434	0.500000
3	245873	0.408248
4	453177	0.408248

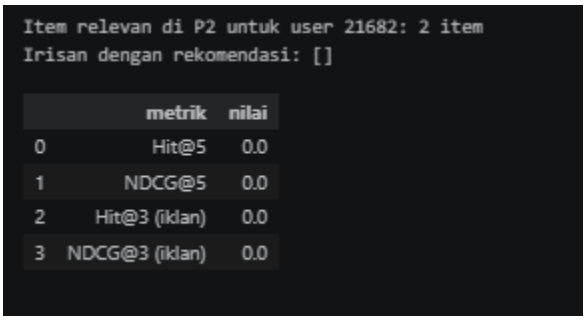
Item yang diiklankan: [230397, 180434, 245873]

Gambar 18

Jadi, tiga item yang diiklankan adalah 230397, 180434, dan 245873.

b. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:

- Hit@5
  Jawab: 0, karena tidak ada item rekomendasi yang sama dengan item yang diinteraksi user pada P2.
- NDCG@5
  Jawab: 0, karena tidak ada item relevan yang berhasil direkomendasikan, sehingga tidak ada nilai dari posisi peringkat.



Gambar 19

Evaluasi dilakukan pada user 21682. Pada periode P2 (September 2015), user ini berinteraksi dengan 2 item. Item-item itu dibandingkan dengan item hasil rekomendasi KNN, dan hasilnya tidak ada satu pun yang sama. Alasan kenapa hasilnya 0, karena: P2 berjarak sekitar 2,5 bulan dari P1, jadi minat user bisa sudah berubah. Data sangat jarang, user ini hanya punya 2 item di data training sehingga tetangga KNN-nya sedikit. Dan Jumlah item sangat banyak (sekitar 81 ribu), jadi peluang rekomendasi tepat sasaran sangat kecil. Nilai ini bukan karena kode atau perhitungan salah, tapi karena keterbatasan data.